Tarea: Planificación de Rutas de Drones con Zonas de Exclusión

Teoría de la Computación

Martín Vidal
Ingeniería de Sistemas
Universidad ORT Uruguay
Montevideo, Uruguay
Nro. Est. 68694

***Abstract*—El presente informe desarrolla la resolución para la tarea planteada como entrega final del curso Teoría de la Computación. Su objetivo principal es analizar la complejidad computacional de dos problemas específicos (SAT y Planificación de Rutas de Dron), y demostrar la relación entre ellos mediante técnicas de verificación y reducción polinomial. Asimismo, adjunto a este documento se incluye una implementación en Haskell, tanto de los solvers como de los verificadores de los problemas indicados.**

I. Introducción

El estudio de la complejidad computacional permite clasificar problemas según los recursos necesarios para resolverlos y comprender sus relaciones mediante reducciones polinomiales. En el contexto de esta tarea, se analizan dos problemas específicos: A (SAT), representación clásica del problema de satisfacibilidad booleana, y B, un problema de planificación de rutas para un dron sujeto a restricciones de distancia, prioridades y exclusión entre zonas.

El propósito principal es demostrar que el problema B es NP-Completo, para lo cual se divide este docuemento (y la tarea) en distintas secciones. Primero, se muestran las definiciones de los dominos y se implementan verificadores (código incluido en la entrega) que permiten comprobar en tiempo polinomial si una solución candidata es válida. Luego, se establece que tanto A como B pertenecen a la clase NP. Seguidamente, se muestra una reducción polinomial desde SAT hacia B, empleando como recurso intermedio el problema TSP (Traveling Salesman Problem). Esta estrategia permite reutilizar construcciones clásicas de la teoría, componiendo reducciones que muestren la equivalencia enre ambas instancias. Es decir, mostrar que, una fórmula SAT es satisfacible si y solo si la instancia correspondiente del problema B admite una ruta válida.

Asimismo, el informe presenta ejemplos concretos de fórmulas SAT, permitiendo validar la corrección de los verificadores y solvers implementados.

La última sección hace referencia a la implementación en otros modelos vistos en clase como ser, Imp, χ y Máquinas de Turing.

II. Marco Teórico

Este trabajo se enmarca dentro del área de la complejidad, uno de los pilares de la teoría de la computación que estudia como intentar responder a preguntas como: ¿Qué tan difícil es resolver un problema? ¿Cuánto tiempo necesitaría un algoritmo para verificar o encontrar una solución? ¿Que relación existe entre la complejidad de un problema y otro?

Para esto es necesario entender algunos conceptos esenciales que se explican a continuación. En tal sentido, reconocemos que es posible que los conceptos carezcan de la formalidad requerida, tanto en esta sección como en las secciones subsiguientes. Esperamos, que la falta de rigurosidad se compense con una mayor claridad.

A. Problemas de decisión

Un problema de decisión es un tipo especial de problema computacional cuya respuesta siempre es Sí o No. No pide construir una solución completa ni encontrar un objeto particular: solo pregunta si existe o no existe algo que cumpla ciertas condiciones. Es el tipo de problemas más usado en complejidad computacional porque simplifica el análisis y permite comparar problemas entre sí, asimismo es realtivamente sencillo vincularlos con la definición de los conceptos P, NP y NP-Completo.

B. Clase P y Clase NP

En complejidad, las clases P y NP son fundamentales para entender qué tan difícil es resolver distintos problemas. Aunque a veces se presentan como conceptos abstractos, su idea principal es bastante simple: medir cuánto tiempo tarda un algoritmo en resolver o verificar un problema, en función del tamaño de la entrada.

- La clase P contiene a todos los problemas de decisión que pueden resolverse completamente en un tiempo que crece de forma polinomial con respecto al tamaño de la entrada. Intuitivamente, diríamos que es el tipo de problemas que se pueden resolver ”rápidamente” o al menos en un tiempo razonable. Algunos problemas que caen dentro de esta clase podrían ser: ordenar una lista

de números o encontrar el camino mas corto entre dos vértices de un grafo.

- La clase NP contiene a todos los problemas de decisión para los cuales una solución candidata puede ser verificada rápidamente, aunque no se sepa cómo encontrarla rápidamente. Más formalmente, se definen como un grupo de problemas en los cuales, dada una solución candidata, es posible verificar si dicha solución es correcta en tiempo polinomial. Es importante entender que NP no significa "no polinomial", tampoco implica que no se pueda resolver; NP significa "Tiempo Polinomial NO Determinista". Si bien se hace esta distinción para caracterizar a los problemas P y NP; teniendo en cuenta que estos últimos se definen en relación al tiempo que toma verificar una solución, se desprende que los problemas P también son NP. En el sentido que se verifican las soluciones candidatas en tiempo polinomial. (En el caso de problemas P, se puede usar el propio solver para verificar la posible solución). De hecho, no se ha demostrado si $P = NP$ o si $P \neq NP$.

Los problemas tratados en este documento corresponden a la clase NP.

C. Problemas NP-Completos

Un problema NP-Completo tiene dos características:

- Pertenece a NP (sus soluciones se pueden verificar en tiempo polinomial
- Es al menos tan difícil como cualquier otro problema de NP. Esto significa que si se encontrase un algoritmo eficiente para resolver cualquier NP-Completo, entonces todos los problemas de NP tendrían una solución eficiente.

Existe una cantidad importante de problemas clásicos NP-Completos conocidos. Uno de los objetivos de este documento es demostrar que el problema B de las rutas de los drones es también NP-Completo.

D. Algoritmos verificadores

En lugar de preocuparse por cómo encontrar una solución, muchas veces se estudia qué tan difícil es verificar si una solución propuesta es correcta. Un verificador toma: una instancia del problema y una solución candidata y responde si esa solución es válida. En este trabajo se implementaron: verifyA para el problema SAT y verifyB para el problema de planificación de rutas de drones. (Referirse al código adjunto)

La idea es demostrar que verificar estas soluciones puede hacerse en tiempo razonable (polinomial), incluso aunque encontrar dichas soluciones puedan resultar ser muy difíciles de hallar.

E. Reducciones Polinomiales

Si bien intuitivamente se puede decir que un problema es dificil. Se requiere cierto formalismo para mostrar que un problema es NP-Completo. Hay que compararlo con un problema NP-Completo conocido. Para esto se introduce el concepto de reducción polinomial. Una reducción polinomial transforma una instancia de un problema NP-Completo en una instancia equivalente de otro otro problema y esa transformación deberá realizarse en tiempo polinomial.

F. Tiempo Polinomial

A lo largo de este documento se menciona "tiempo polinomial" en múltiples oportunidades. A modo de clarificar el concepto, indicar que se utiliza en el sentido que el tiempo que toma ejecutar un algoritmo puede acotarse por una función del tipo:

$$T(n) = n^k$$

Donde n es el tamaño de la entrada y k una constante fija. Se suele aceptar que si el tiempo crece como una potencia de n (tamaño de la entrada), el algoritmo se considera eficiente, al menos en relación a los tiempos no polinomiales como podría ser un algoritmo de crecimiento exponencial. En cualquier caso, lo que se considere, rápido y/o eficiente dependerá de las condiciones propias de cada problema tratado.

III. Diseño, Implementación y Reducción

Esta sección abarca el trabajo propiamente llevado adelante para esta tarea. Se describen los problemas, las principales estructuras para resolverlos y verificarlos, asi como la estrategia para reducir de un problema a otro.

A. Problemas a estudiar

La tarea plantea el estudio de dos problemas. Por un lado, un problema NP-Completo, clásico, SAT. Esto es, dada una fórmula booleana en forma normal conjuntiva (CNF), decidir si existe una asignación de valores de verdad a sus variables que haga que la fórmula sea verdadera. Por otra parte, un problema de planificación de rutas de drones con zonas de exclusión. El objetivo es determinar si una ruta factible que cumpla con una serie de restricciones impuestas, como ser, completar un circuito volviendo a la base, recorrido limitado por la autonomía del dron, completar un mínimo de prioridades, y considerar las zonas mutuamente excluyentes para definir la ruta. En definitiva, la pregunta de decisión es si existe una ruta que cumpla todas las condiciones anteriores.

B. Definición de Dominios y Estructuras de Datos

Específicamente, la tarea requiere una implementación tanto de los verificadores como de los solvers, en Haskell. La idea no es replicar el código en el documento, pero sí, explicar las decisiones que se tomaron para modelar el problema con las estructuras del lenguaje.

1) Problema A (SAT): El dominio de A es finito y binario. Los únicos elementos con los que se trabaja son variables booleanas y operadores lógicos (conjunción, disyunción y negación). La estructura es una forma normal conjuntiva que organiza la fórmula de una manera específica. Esto es:

- Literal: Una variable booleana o su negación (ej: x, $\neg y$)
- Cláusula: Una disyunción (operador $\vee$, "OR") de uno o más literales (ej: $(x \vee \neg y \vee z)$)
- Fórmula: Una conjunción (operador $\wedge$, "AND") de múltiples cláusulas (ej: $(x \vee \neg y \vee z) \wedge (\neg x \vee y) \wedge (\neg z)$).

Es así que se puede representar estos elementos en Haskell de la siguiente manera:

- El dominio serán las fórmulas
- Cada fórmula será una lista de cláusulas
- Las cláusulas serán listas de literales
- Los literales serán una pareja (nombre (String), positivo/negativo(Bool))

Están implicitas las disyunciones dentro de las cláusulas y las conjunciones ente cláusulas. De esta manera, cada una de las soluciones será una lista de literales, donde habrá una asignación para cada una de las variables que aparecen en la fórmula.

2) Problema B (Planificación de rutas de drones): El problema B se modeló a partir de la especificación de la entrada formal proporcionada en la letra. De esta manera, tenemos:

- Un conjunto de puntos de control (que representan los puntos visitados en la ruta).
- Una matriz que indica la distancia que existe entre cada par de puntos.
- Una función que indica la distancia de un punto dado a la base (inicio y final del recorrido)
- Un conjunto de pares de exclusion que indica las zonas que son mutuamente excluyentes.
- Una función de prioridad que indica la importancia de un punto específico para el recorrido.
- La prioridad mínima requerida que debe ser alcanzada.
- El límite de autonomía del dron

La idea es mantener la representación de los elementos del problema de la manera más sencilla posible. En tal sentido, se optó por diseñar el dominio con los tipos de datos más sencillos de manejar. Es así entonces:

- El dominio contiene todos los parámetros descriptos, esto es: la lista de puntos de control, las distancias entre puntos, las distancias a la base, las exclusiones, las prioridades, la prioridad minima y la autonomía máxima.
- Los puntos serán una lista de enteros.
- La matriz de distancias será una matriz de enteros donde los indices indican los puntos de control y los valores, la distancia entre estos.
- Las zonas de exclusión mutua se representará con una lista de parejas punto de control/punto de control (enteros).
- Las prioridades es una lista de punto de control/prioridad (entero/entero).
- Un entero para la prioridad mínima
- Un entero para la autonomía.

Cada una de las soluciones del problema se expresará como una lista de puntos de control.

C. Verificadores en tiempo polinomial

Como se mencionó anteriormente, los verificadores tienen como objetivo comprobar si una solución candidata satisface la condiciones de un problema. En esta seccion en particular, se mostrará como esta verificación se realiza en tiempo polinomial mostrando los órdenes de ejecución de las funciones involucradas.

1) VerifyA: Para calcular de forma precisa el orden de ejecución de verifyA, se requiere conocer el tamaño de la entrada (la fórmula y la solución) y el costo de las operaciones internas. La intención no es calcular rigurosamente sino que basta con comprobar que la complejidad algorítmica de es orden polinómico.

En lugar de reproducir el código aquí y mostrar linea por linea el costo de ejecución, basta saber que verifyA encadena la llamada a otras tres funciones de la siguiente manera:

$$\textbf{verifyA} \rightarrow \textbf{evalClause} \rightarrow \textbf{evalLiteral} \rightarrow \textbf{lookupVar}$$

lookupVar busca una variable dentro de la solución candidata que es una lista de pares string/bool. Esto es de orden *O(N)*, donde N es la cantidad de variables. ***evalLiteral*** hace una única llamada a lookVar por variable, lo que implica que el orden también es *O(N)*. ***evalClause*** usa any sobre una cláusula, aplicando evalLiteral a cada literal. Una cláusula tiene como máximo L literales. De esto se deduce que el orden de ejecucion es *O(L.N)*. ***verifyA*** usa all sobre la formula, aplicando evalClause a cada una de las M cláusulas.

Por lo tanto, es un algoritmo polinómico ya que su complejidad es de orden ***O(M.L.N)***, es decir, está acotada por un polinomio de grado 3, respecto de las variables de entrada.

2) VerifyB: El cantidad de parámetros de entrada para esta función es mayor, por lo cual, la cadena de llamadas es mayor al tiempo que hay invocaciones a funciones auxiliares. En cualquier caso, es de hacer notar, que la función principal ejecuta tres chequeos en secuencia, además de verificar previamente que la ruta no esté vacía.

$$\textbf{verifyB} \rightarrow \textbf{violatesExclusions} \rightarrow \textbf{any} \rightarrow \textbf{elem (en route)}$$

Chequeo de exclusiones: ***violatesExclusions*** utiliza any para iterar sobre la lista de exclusiones (excl). A su vez La función anónima dentro de any llama al método elem dos veces para cada par de exclusión, buscando los puntos en la lista route.

$$\textbf{verifyB} \rightarrow \textbf{totalPriority} \rightarrow \textbf{map} \rightarrow \textbf{lookupPriority}$$

Chequeo de Prioridad: ***totalPriority*** utiliza map para aplicar ***lookupPriority*** a cada punto de la route. lookupPriority es la función base que accede a la estructura de datos Priorities (pr).

$$\textbf{verifyB} \rightarrow \textbf{totalDistance} \rightarrow \begin{cases} \textbf{lookupBaseDist} \rightarrow \text{lookup} \\ \textbf{distBetweenConsecutive} \rightarrow \text{dist} \end{cases}$$

Chequeo de Distancia: ***totalDistance*** llama a ***lookupBaseDist*** dos veces para obtener las distancias de la base al inicio y al final de la ruta. lookupBaseDist accede a la estructura de datos BaseDistances (bd). totalDistance llama a ***distBetween-Consecutive*** para sumar las distancias internas de la ruta. ***distBetweenConsecutive*** llama recursivamente a la función base dist que busca en la DistanceMatrix).

Priorities (pr) y BaseDistances (bd) son listas de pares, lo que implica una búsqueda lineal. lookupPriority y lookup-BaseDist; ambas recorren linealmente una lista de tamaño P,

siendo P el número de entradas en la lista de prioridades o de distancias a la base, por lo que estas funciones son de orden *O(P)*. La condición de verificación de ruta no nula es una operación de tiempo constante, es decir orden *O(1)*. El control de zonas de exclusión mutua, es una iteración que se ejecuta E veces, siendo E el largo de la lista de exclusiones. Por lo tanto el costo de este chequeo es E por la cantidad de puntos de control que incluye la ruta. Si a este largo le llamamos R, entonces el orden de ejecución del chequeo de exclusión de zonas es *O(E.R)*. El costo por sumar las prioridades de cada punto del recorrido, implica ejecutar R veces (cantidad de puntos del recorrido) map y sum. Asimismo, como se indicó el costo para hacer el lookup de las prioridades P, el numero de entradas de la lista de prioridades. De esto se desprende, que el costo de ejecución de este chequeo es de orden *O(R.P)*. El costo total del cálculo de la distancia recorrida tiene tres componentes; la distancia de la base al primer punto, el costo de las llamadas recursivas a la funcion distBetweenConsecutive que obtiene la distancias entre puntos consecutivos y se le suma el costo de obtener la distancia desde el ultimo punto del recorrida a la base. El único costo relevante es las R-1 llamadas recursivas necesarias para calcular la distancias intermedias de todos los puntos del recorrido. Si se consideran todos los chequeos, el costo de verifyB será el costo de la operación más cara entre las cuatro condiciones.

$$\text{Costo(verifyB)} = \max\{O(1), O(E \cdot R), O(R \cdot P), O(R + P)\}$$

Independientemente de las consideraciones que pudieran hacerse respecto a los distintos parámetros de entrada, algo que permitiera afinar mejor el verdadero costo de esta función; es claro que el orden de ejecución es polinómico.

D. Solvers en tiempo exponencial

La implementación de las funciones:

$$\textbf{solveA :: DomA} \rightarrow \textbf{SolA}$$

$$\textbf{solveB :: DomB} \rightarrow \textbf{SolB}$$

se encuentran en el archivo de Haskell adjunto a esta entrega. En cualquier caso, se puede sacar ciertas conclusiones respecto de orden de ejecución, simplemente analizando su estructura. Es decir, dado un DomA (la fórmula en CNF), solveA debe:

- Obtener todas las variables de la fórmula.
- Generar todas las posibles asignaciones (valuaciones) para esas variables. De esto se deduce que, si el número de variables es n, entonces habrá 2^n valuaciones.
- Para cada valuación (cada SolA posible), se debe ejecutar verifyA.
- Eventualmente el resultado será True o False dependiendo si alguna valuación satisface la fórmula.

El hecho que tener que generar y probar todas las valuaciones, y que estas dependan exponencialmente de la entrada, muestra que su ejecución es en tiempo exponencial.

Asimismo, esta misma estrategia de fuerza bruta se utiliza para implementar solveB. Considerar que, la función, construye todas las rutas posibles, haciendo todas las combinaciones de zonas disponibles en todos los subconjuntos por tamaños crecientes y a su vez todas las permutaciones de puntos dentro de cada ruta considerada. Si bien, corta la ejecución cuando encuentra la primera ruta válida, y por otra parte, prueba con rutas cortas, primero; surge de un análisis básico que este algoritmo se ejecuta en tiempo exponencial. Se podría considera implementar mejoras en el rendimiento, como ser, podas por distancia o prioridad parcial, eliminando rutas (y cálculos) por exceder distancias o no alcanzar prioridad mínima; en cualquier caso nunca podría ejecutarse en tiempo polinomial.

E. Reducción polinomial entre problemas

En esta sección presentamos la argumentación formal necesaria para demostrar que el Problema B es NP-Completo, siguiendo la estructura clásica:

- Mostrar que B $\in$ NP, es decir, que las soluciones candidatas pueden ser verificadas en tiempo polinomial.
- Construir una reducción polinomial desde un problema NP-Completo conocido hacia B.

1) B $\in$ NP: Como se detalló anteriormente, el verificador del problema B (verifyB) comprueba en tiempo polinomial: la distancia total de la ruta (suma lineal en el largo de la ruta), la suma de prioridades, la ausencia de repeticiones, y el cumplimiento de las exclusiones. Ninguna de estas operaciones requiere más que tiempo proporcional al número de puntos de la ruta o al tamaño explícito de la entrada, por lo que verifyB corre en tiempo polinomial, y por tanto B $\in$ NP.

2) Estrategia de reducción SAT $\rightarrow$ B: Para mostrar que B es NP-Completo necesitamos construir una reducción: $SAT \leq_p B$ En clase se sugirió que hacer la reducción directamente entre estos problemas podría resultar muy complicada y por tanto se recomendó hace uso de una estrategia de dos pasos, intercalando una reduccion intermedia desde SAT hacia otro problema NP-Completo conocido, y desde este último al problema B. La idea es hacer la reducción desde un problema NP-Completo, conocido, cuyas características sean tales, que la reducción se presente más sencilla que hacerla desde SAT directamente. En tal sentido, hay que realizar lo siguiente:

- Encontrar ese tercer problema intermedio que muestre ser mejor que SAT para hacer la reducción.
- Construir la reducción propiamente dicha, con ese problema.
- Mostrar que existe tambien una reduccion desde SAT hacia el problema intermedio
- Justificar la transitividad entre relaciones polinomiales.

Considerar realizar la reducción directamente mostraba ciertas dificultades para encontrar el paralelismo entre problemas. Parecía razonable representar los literales como los puntos de control del problema B; pero el resto de los elementos parece más forzados. La sugerencia de cambiar de estrategia, llevó a considear otras opciones, por ejemplo Orienteering (que no estaba dentro de la lista de problemas clásicos publicada en Aulas, pero que fue sugerido por una IA). Menos probable

aun, resultó ser, la consideración de Clique; aunque habían elementos que podía mapearse entre problemas; por ejemplo, en B se busca un conjunto de puntos que cumpla restricciones, mientras en Clique se busca un conjunto de vértices que cumplan una propiedad; o modelar las aristas que no existen en el grafo como exclusiones. Nuevamente la sugerencia de utilizar TSP definió el camino a seguir. El TSP se presenta en numerosas variantes, entre ellas el TSP clásico (ciclo), el TSP asimétrico, el TSP métrico, TSP-PATH, TSP con ventanas de tiempo y versiones con ganancias como prize-collecting. En este informe se utiliza la versión clásica de decisión con ciclo Hamiltoniano, que es la forma estándar empleada en reducciones NP-Completas. En definitiva la estrategia será la siguiente:

$$\text{SAT} \leq_p \text{TSP} \leq_p B.$$

Como se justificará más adelante, gracias a la transitividad de las reducciones polinomiales, esto permitirá concluir que SAT $\leq_p B$.

La existencia de una reducción desde SAT a TSP está ampliamente documentada. Sin embargo, las referencias parecen contradecirse. Naturalmente no estoy en condiciones de identificar cual es la respuesta correcta, pero dejo ambas argumentaciones y referencias.

Por un lado, se indica que esta reducción es excepcionalmene compleja y requiere un encadenamiento de reducciones intermedias que involucran a múltiples problemas NP-Completos intermedios[1]:

$$\text{SAT} \leq_p \text{3-SAT} \leq_p \text{Clique}$$

$$\leq_p \text{Cobertura de vértices (Vertex Cover)}$$

$$\leq_p \text{Ciclo Hamiltoniano} \leq_p \text{TSP}$$

Por otra parte, hay referencias que muestran un camino más directo con la siguiente idea [2]: Dada una fórmula φ en CNF, la reducción estándar construye un grafo completo cuyas distancias están definidas de modo que cualquier ciclo hamiltoniano de costo menor o igual a un cierto valor K corresponde a una asignación satisfactoria de φ. Si φ es satisfacible, existe un ciclo de costo $\leq K$. Si no lo es, ningún ciclo hamiltoniano cumple con dicha cota.

En cualquiera de las argumentaciones anteriores, se asegura que esta reducción es polinomial. Este el dato que realmente nos interesa, desde el punto de vista formal, para justificar la utilización de TSP como reducción intermedia. Desde el punto de vista práctico, el Problema del Viajante (TSP) pertenece a la clase de problemas clásicos NP-Completos y su estructura basada en recorridos y costos resulta particularmente apropiada para transformarlo luego en una instancia de B. Es decir es válida esta reducción polinómica:

reduceSATtoTSP :: **DomA** $\rightarrow$ **TSPInstance**.

De esta manera, ahora solo resta encontrar la reducción de TSP a B para justificar y hacer la composición de reducciones polinomiales.

<u>Reducción TSP $\rightarrow$ B</u>

Sea una instancia TSP formada por:

$$C = \{c_1, \ldots, c_n\}, \qquad d(c_i, c_j), \qquad K.$$

Construimos una instancia de B como sigue:

- Puntos:
$$P = \{\text{Base}, p_1, \ldots, p_n\}.$$
- Matriz de distancias:
$$D(p_i, p_j) = \begin{cases} d(c_i, c_j) & \text{si } i \neq j, \\ \text{MaxNum} & \text{si } i = j. \end{cases}$$
MaxNum es un artificio, para evitar que se seleccione como válida una distancia desde un punto de control a si mismo. Además,
$$D(\text{Base}, p_i) = D(p_i, \text{Base}) = 0.$$
- Distancia a la base:
$$r(x) = 0 \quad \forall x \in P.$$
- Exclusiones: $E = [\ ]$.
- Prioridades: $b(p_i) = 1$ y $b(\text{Base}) = 0$.
- Prioridad mínima: $V = n$.
- Límite de distancia: $M = K$.

Asi, esta construcción fuerza a cualquier solución válida de B a:

1) visitar los n puntos sin repetición (debido a MaxNum);
2) completar un ciclo que inicia y termina en Base;
3) tener costo total $\leq K$;
4) acumular prioridad exactamente n.

Por lo tanto, cualquier ruta válida corresponde a un ciclo hamiltoniano de la instancia TSP original.

reduceTSPtoB :: **TSPInstance** $\rightarrow$ **DomB**.

Es de hacer notar, que la manera en la cual se construyen los elementos de la reducción, garantiza que la misma sea polinómica. Es decir, la reducción consiste en transformar una instancia de (C,d,K), esto es: conjunto de ciudades, matriz de distancias y límite de costo del ciclo; en una instancia del problema B, tal como se mostró. Si todo esto puede hacerse en tiempo polinómico respecto al tamaño de la entrada, entonces la reducción es polinómica. De hecho esto es asi:

- La construcción del conjunto de puntos, requiere crear n+1 elementos, lo cual es O(n)
- La matriz de distancias solo agrega una fila y columna para la base, esto implica que copiarla es de O(n^2)
- Las distancias a la base, se definen en 0, para cada uno de los puntos, esto se hace en O(n)
- Las prioridades, se establece el valor 1 para cada punto, esto es en orden O(n)
- Las exclusiones es una lista vacia, esto es O(1)
- Es orden O(1) tanto para el valor de prioridad mínima a alcanzar, como el de autonomía del dron

La construcción completa puede hacerse en orden O(n^2). Esto justifica que la reducción sea polinómica.

Por último indicar que las reducciones polinómicas son transitivas. Supongamos que $A \leq_p C$ mediante una función f computable en tiempo polinómico y que $C \leq_p B$ mediante una función g, también polinómica. Podemos entonces construir una función compuesta

$$h = g \circ f$$

que transforma directamente instancias de A en instancias de B.

Como la ejecución de h consiste simplemente en aplicar primero f y luego g, su tiempo total está acotado por la suma de dos polinomios. Dado que la suma de funciones polinómicas sigue siendo polinómica, se concluye que h también es computable en tiempo polinómico. Por lo tanto, si $A \leq_p C$ y $C \leq_p B$, entonces

$$A \leq_p B.$$

Se ha demostrado que:

1) $B \in$ NP (posee un verificador polinomial).
2) SAT $\leq_p B$ mediante la composición SAT $\to$ TSP $\to$ B.

Como SAT es NP-Completo, se concluye:

$$\boxed{B \text{ es NP-completo}}.$$

IV. Ejemplos de instancias de prueba

En este apartado se presentan dos instancias simples del problema SAT, una satisfacible y otra insatisfacible, y se muestra con qué instancias del Problema B se corresponden luego de aplicar la estrategia de reducción establecida en la sección anterior (SAT $\to$ TSP $\to$ B).

A. Ejemplo 1: Fórmula satisfacible

Considerar la siguiente fórmula:

$$(x_1 \vee \neg x_2 \vee x_3) \ \wedge \ (\neg x_1 \vee x_2 \vee x_3) \ \wedge \ (x_2 \vee x_3).$$

Esta fórmula es satisfacible; por ejemplo, una valuación posible es:

$$x_1 = \text{True}, \qquad x_2 = \text{True}, \qquad x_3 = \text{True}.$$

Aplicando la reducción SAT $\to$ TSP, se obtiene una instancia del TSP con un conjunto de ciudades $C = \{c_1, c_2, c_3\}$ y una matriz de costos construida de manera que existe un ciclo hamiltoniano de costo $\leq K$ si y sólo si la fórmula es satisfacible.

A continuación, se construye la instancia del Problema B mediante la reducción TSP $\to$ B. Los elementos de la instancia resultante son:

- Puntos:
$$P = \{\text{Base}, p_1, p_2, p_3\}.$$
- Matriz de distancias: para $i \neq j$,
$$D(p_i, p_j) = d(c_i, c_j), \qquad D(p_i, p_i) = \text{MaxNum},$$
y
$$D(\text{Base}, p_i) = D(p_i, \text{Base}) = 0.$$
- Distancia a la base: $r(x) = 0$ para todo $x \in P$.
- Exclusiones: $E = [\ ]$.
- Prioridades: $b(p_i) = 1$ y $b(\text{Base}) = 0$.
- Prioridad mínima: $V = 3$.
- Límite de distancia: $M = K$, donde K proviene de la instancia TSP.

Dado que la fórmula SAT es satisfacible, la instancia correspondiente del TSP posee un ciclo de costo $\leq K$, y por lo tanto existe en B una ruta válida de la forma:

$$\text{Base} \to p_{i_1} \to p_{i_2} \to p_{i_3} \to \text{Base},$$

cumpliendo distancia $\leq M$ y prioridad total exactamente 3. Por lo tanto, la instancia de B asociada es satisfacible.

B. Ejemplo 2: Fórmula insatisfacible

Considerar la fórmula:

$$(x_1) \ \wedge \ (x_2 \vee x_3) \ \wedge \ (\neg x_1).$$

Esta fórmula es insatisfacible, ya que obliga simultáneamente a $x_1 = \text{True}$ y $x_1 = \text{False}$.

Aplicamos las reducciones SAT $\to$ TSP $\to$ B. La instancia de TSP correspondiente no tendrá ningún ciclo hamiltoniano de costo $\leq K$.

Aplicando la reducción TSP $\to$ B, obtenemos:

- Puntos:
$$P = \{\text{Base}, p_1, p_2, p_3\}.$$
- Matriz de distancias:
$$D(p_i, p_j) = d(c_i, c_j), \quad i \neq j; \qquad D(p_i, p_i) = \text{MaxNum},$$
y
$$D(\text{Base}, p_i) = D(p_i, \text{Base}) = 0.$$
- Exclusiones: $E = [\ ]$.
- Prioridades: $b(p_i) = 1$, $b(\text{Base}) = 0$.
- Prioridad mínima: $V = 3$.
- Límite de distancia: $M = K$.

En este caso, como la fórmula SAT original era insatisfacible, la instancia TSP asociada tampoco admite un ciclo de costo $\leq K$. Por lo tanto, ninguna ruta posible en el Problema B logrará visitar los tres puntos $\{p_1, p_2, p_3\}$ sin incurrir en costo mayor a M, dado que recorrer todos ellos es necesario para alcanzar la prioridad mínima $V = 3$.

Se concluye entonces que la instancia de B correspondiente al segundo ejemplo de SAT es insatisfacible.

C. Evaluación experimental

Con la intención de obtener algún tipo de evidencia empírica que reafirmara la diferencia entre el orden de ejecucion de los solvers y los verifier, se incluyen en el código adjunto un número de ejemplos. A decir verdad, es dificil apreciar la diferencia en los tiempos de ejecución si no se tunean apropiadamente las pruebas. En primer lugar, armar los ejemplos con un buen número de variables. En el código hay un ejemplo

con 20 variables de prueba. Aun asi, no es fácil apreciar la diferencia en los tiempos de ejecución. El otro elemento importante para generar esa diferencia es diseñar los ejemplos para que el solver explore la mayor cantidad de soluciones posibles. Preferentemente, utilizar una fórmula insatisfacible para garantizar que intente con todas las soluciones candidatas. Durante las pruebas se intentó utilizar PerfMon para capturar el uso de memoria de allvaluations, pero el hecho que la ejecución se realice dentro de la memoria asignada al IDE, hace el uso específico de memoria de los procesos particulares quede enmascarado dentro de la memoria total asignada. Probablemente se pudiera utilizar otras herramientas mas adecuados, pero se entendió que esto estaba por fuera del alcance de la tarea. En cualqueir caso, si se agrega la salida por consola de cada solución que se está probando, la diferencia entre los tiempos del solver y el verifier se hacen evidentes. (Se incluye dentro del código una version hecha con IA, que muestra la salida por consola).

V. Máquina de Turing

En esta sección especificamos la reducción entre instancias de SAT y del Problema B mediante una Máquina de Turing. Para ello se define un alfabeto, una codificación de las instancias de ambos dominios y una Máquina de Turing que, dada una codificación de SAT, produce la codificación correspondiente de B.

A. Alfabeto

Definimos un alfabeto Σ para representar fórmulas SAT, matrices, listas y estructuras del Problema B. El alfabeto queda definido de la siguiente forma:

$$\Sigma = \{0, 1, @, \$, ;, :, (,), p, B, \ , -, \#\},$$

donde:

- 0,1: dígitos binarios para representar números y pesos.
- @: separador entre secciones de una instancia
- $: separador entre cláusulas o entre elementos de listas.
- ;: separador de filas en matrices.
- ,: separador entre elementos dentro de una fila o lista.
- (:): opcionales para delimitar literales o pares.
- p, B: etiquetas para puntos y la base en el Problema B.
- -: marca para listas vacías (por ejemplo para exclusiones).
- #: símbolo de blanco de la cinta de la Máquina de Turing.

B. Codificación de instancias de SAT

Representamos una fórmula como una lista de cláusulas separadas por "$" y cada cláusula es una lista de literales separados por comas. Cada literal se codifica como un par (s, i) donde $s \in \{0, 1\}$ indica el signo (1: positivo; 0: negado) e i es el índice de variable en binario.

La codificación completa de una instancia de SAT será:

$$SAT@C_1\$C_2\$\cdots\$C_m.$$

C. Codificación de instancias de B

Codificamos una instancia del Problema B con el siguiente formato:

B @ P @ D @ r @ b @ E @ M @ V,

donde:

- P: lista p_1,p_2,...,p_n.
- D: matriz en formato de filas separadas por ";" y columnas por ",".
- r: lista de distancias a la base, separadas por ",".
- b: lista de prioridades, separadas por ",".
- E: lista de exclusiones; si está vacía, se escribe "–".
- M: límite máximo de distancia en binario.
- V: prioridad mínima requerida en binario.

D. Máquina de Turing para la reducción SAT → B

La Máquina de Turing $\mathcal{M}$ actúa sobre la codificación de SAT y produce la correspondiente instancia de B aplicando la reducción compuesta:

$$\text{SAT} \xrightarrow{\texttt{reduceSATtoTSP}} \text{TSP} \xrightarrow{\texttt{reduceTSPtoB}} \text{B}.$$

A grandes rasgos el procesamiento sería de la siguiente manera:

a) Lectura y análisis sintáctico: La MT recorre la cadena de entrada, identifica cada cláusula y cada literal, y escribe al final de la cinta (luego del primer blanco a la derecha) la lista de cláusulas con índices binarios. Esta operación es lineal en el tamaño de la entrada.

b) Construcción de la instancia TSP: Esta fase implementa la reducción SAT $\leq_p$ TSP. Para cada variable y para cada cláusula se escriben en la cinta estructuras de nodos y aristas cuyos pesos codificados en binario garantizan que existe un ciclo hamiltoniano de costo $\leq K$ si y sólo si la fórmula SAT es satisfacible. El tamaño del grafo producido es polinómico en la longitud de la fórmula.

c) Construcción de la instancia B: La MT lee la codificación del TSP construida en la fase anterior y escribe:

- la lista de puntos $P = \{\text{Base}, p_1, \ldots, p_n\}$;
- la matriz D copiando la matriz de costos del TSP y asignando $D(p_i, p_i) = \text{MaxNum}$ y $D(\text{Base}, p_i) = 0$;
- la función r con todos valores 0;
- la lista de prioridades $b(p_i) = 1$ y $b(\text{Base}) = 0$;
- la lista de exclusiones $E = -$;
- los valores $M = K$ y $V = n$.

Todas estas operaciones consisten en copiar, extender o traducir estructuras de tamaño polinómico.

d) Resultado final.: La MT escribe la salida final según el formato B @ P @ D @ r @ b @ E @ M @ V, deja el cabezal al inicio y se detiene.

En a) la operación es el tamaño de la entrada. En b) se reproduce la construcción clásica SAT → TSP, cuyo tamaño es polinómico. En c) copia y adapta estructuras de tamaño cuadrático en n (número de variables o ciudades), lo cual

sigue siendo polinómico. Por lo tanto, la Máquina de Turing completa opera en tiempo polinómico en relación al tamaño de la instancia SAT original.

Resumiendo: Sea φ la fórmula SAT de entrada y sea $B = \mathcal{M}(\varphi)$ la instancia final producida.

- La reducción SAT $\rightarrow$ TSP asegura que φ es satisfacible si y sólo si la instancia TSP intermedia admite un ciclo de costo $\leq K$.
- La reducción TSP $\rightarrow$ B garantiza que dicho ciclo existe si y sólo si en B existe una ruta válida que visita todos los puntos sin repetición, tiene costo $\leq M = K$ y suma de prioridades $V = n$.

Esto muestra que la MT construida preserva satisfacibilidad

CONCLUSIONES

En primer lugar, mencionar que la tarea requería en su segunda parte, además de la codificación en de la reducción en Máquina de Turing, implmentar el verificador de SAT en Imp y el verificador de B en χ. Estas dos últimas no fueron realizadas.

En este trabajo se abordó el estudio de la complejidad computacional a través de la construcción y análisis de dos problemas: el Problema A (SAT) y el Problema B (planificación de rutas de drones considerando restricciones de rutas con prioridades, distancias y exclusiones). A partir de estas definiciones se desarrollaron verificadores para cada problema, explorando las diferencias entre la verificación polinómica y la búsqueda exhaustiva de soluciones. Lo cual permitió mostrar hasta cierto punto, de manera práctica, la distinción fundamental entre las clases P y NP.

Posteriormente se construyó una reducción polinómica desde SAT hacia B. Para ello se empleó una reducción intermedia a TSP, lo que permitió aprovechar estructuras clásicas y bien estudiadas dentro de la teoría. Este enfoque mostró cómo transformar una fórmula booleana en CNF en una instancia TSP equivalente, y luego cómo traducir esa instancia en una instancia válida del Problema B. La composición de ambas reducciones garantizó que el proceso completo fuera polinómico, reflejando la transitividad de las reducciones entre problemas NP.

Además, se formalizó la reducción mediante una Máquina de Turing, definieron los alfabetos y las codificaciones necesarias, y se describió a groso modo el proceso, transformación y producción final.

En resumen, el desarrollo realizado permitió comprender cómo se establecen relaciones de complejidad entre problemas, cómo se justifican dichas relaciones mediante reducciones y cómo estas construcciones pueden implementarse en modelos formales.

REFERENCIAS

[1] Karp, Richard M. Título: "Reducibility Among Combinatorial Problems"Publicación: Complexity of Computer Computations, Proceedings of a Symposium (1972) Páginas: pp. 85-103DOI: $10.1007/978-1-4684-2001-2_9$ https://en.wikipedia.org/wiki/Karp

[2] Garey & Johnson Título: "Computers and Intractability" (1979) https://www.cs.colostate.edu/anderson/cs420/notes/NPCompletenessExamples.pdf